บทบรรยาย — Decision Tree (ต้นไม้ตัดสินใจ)

สำหรับสไลด์ Decision_tree_slides.html · 20 สไลด์ · เวลารวมประมาณ 20 นาที · MSC DPML 2026

วิธีใช้เอกสารนี้

- เอกสารนี้เป็นบทพูดฉบับเต็ม อ่านตามได้ทันทีโดยไม่ต้องเรียบเรียงเพิ่ม เนื้อหาแต่ละหัวข้อตรงกับสไลด์ตามเลขที่กำกับไว้
- บรรทัดที่ขึ้นต้นด้วยเครื่องหมาย ▶ คือจังหวะที่ต้องลงมือทำอะไรกับสไลด์ เช่น กดปุ่มดูการคำนวณ ไม่ต้องอ่านออกเสียง
- เวลาที่กำกับไว้เป็นเวลาโดยประมาณเมื่อพูดด้วยจังหวะปกติ หากเวลาไม่พอ หัวข้อที่ตัดได้โดยไม่เสียใจความคือสไลด์ 16 และ 18

ผังเวลาโดยสังเขป

สไลด์	หัวข้อ	เวลา	สะสม
1	หน้าปก — Decision Tree	0:40	0:40
2	แผนการเดินทาง	0:40	1:20
3	Decision Tree คืออะไร	1:10	2:30
4	โมเดลมี 2 เฟส	0:50	3:20
5	ชุดข้อมูลฝึกสอน	1:10	4:30
6	ควรถามอะไรก่อน	0:50	5:20
7	Info(D) วัดความไม่แน่นอนของข้อมูล	1:40	7:00
8	แบ่งด้วย A แล้วเหลือความไม่แน่นอนเท่าไร	1:10	8:10
9	Information Gain	1:10	9:20
10	ขั้นที่ 1 แบ่งด้วย age	1:10	10:30
11	ขั้นที่ 2 ถึง 3 หา Gain ของ age	1:10	11:40
12	คำนวณครบทั้ง 4 attribute	1:30	13:10
13	age ได้เป็น Root Node	0:40	13:50
14	กิ่งที่ 1 อายุน้อยกว่าหรือเท่ากับ 30	0:55	14:45
15	กิ่งที่ 2 อายุ 31 ถึง 40	0:40	15:25
16	กิ่งที่ 3 อายุมากกว่า 40	0:40	16:05
17	ต้นไม้ที่ได้	0:55	17:00

18	แปลงต้นไม้เป็นกฎ IF-THEN	0:40	17:40
19	ทดลองทำนายข้อมูลใหม่	1:10	18:50
20	สรุป และก้าวต่อไป	1:10	20:00

บทพูดรายสไลด์

สไลด์ 1 — หน้าปก — Decision Tree

เวลาประมาณ 0:40 นาที · สะสม 0:40

สวัสดีครับ วันนี้เราจะมาเรียนเรื่อง Decision Tree หรือต้นไม้ตัดสินใจ ซึ่งเป็นโมเดลจำแนกประเภทที่ใช้กันแพร่หลายที่สุดตัวหนึ่ง

สิ่งที่ผมอยากให้ทุกคนได้กลับไปจากคาบนี้ ไม่ใช่แค่รู้ว่าต้นไม้ตัดสินใจหน้าตาเป็นอย่างไร แต่คือรู้ว่ามันสร้างตัวมันเองขึ้นมาจากข้อมูลได้อย่างไร เราจะคำนวณให้ดูทุกขั้นตอน ด้วยชุดข้อมูลตัวอย่างเล็ก ๆ แค่ 14 แถว ที่ชื่อว่า Who buys computer

ข้อมูลชุดนี้เป็นชุดคลาสสิกครับ ดัดแปลงมาจากตัวอย่าง Playing Tennis ของ Ross Quinlan ผู้คิดค้นอัลกอริทึม ID3 และ C4.5 ซึ่งเป็นสองอัลกอริทึมที่เราจะใช้ในวันนี้

สไลด์ 2 — แผนการเดินทาง

เวลาประมาณ 0:40 นาที · สะสม 1:20

ก่อนเริ่ม ผมขอวางแผนการเดินทางให้ดูก่อนว่าเราจะไปทางไหน

ช่วงแรก เราจะทำความรู้จักต้นไม้ก่อน ว่ามีส่วนประกอบอะไรบ้าง ทำงานกี่เฟส และข้อมูลที่เราจะใช้หน้าตาเป็นอย่างไร

ช่วงที่สอง คือหัวใจของคาบนี้ เราจะสร้างเครื่องมือวัดขึ้นมาหนึ่งตัว ชื่อว่า Information Gain

ช่วงที่สาม เราจะเอาเครื่องมือนั้นไปสร้างต้นไม้จริง แล้วนำไปทำนายข้อมูลใหม่

ทุกอย่างในคาบนี้ตอบคำถามเดียวครับ ถ้ามีคำถามให้เลือกถามหลายข้อ เราจะรู้ได้อย่างไรว่าควรถามข้อไหนก่อน และเราจะตอบด้วยตัวเลข ไม่ใช่ความรู้สึก

สไลด์ 3 — Decision Tree คืออะไร

เวลาประมาณ 1:10 นาที · สะสม 2:30

เริ่มจากรูปร่างก่อนครับ

Decision Tree คือโมเดลจำแนกประเภท ที่มีหน้าตาเป็นผังคำถาม อ่านออกได้ด้วยตาเปล่า ไม่ต้องเป็นนักคณิตศาสตร์ก็เข้าใจได้

มันมีสามส่วน ส่วนแรกคือ internal node หรือโหนดภายใน ซึ่งก็คือคำถามหนึ่งข้อ ถามเกี่ยวกับ attribute หนึ่งตัว อย่างในรูปคือถามว่าอายุอยู่ช่วงไหน

ส่วนที่สองคือ branch หรือกิ่ง หนึ่งกิ่งต่อหนึ่งคำตอบที่เป็นไปได้ ถ้า attribute นั้นมีค่าเป็นไปได้สามค่า ก็แตกออกมาสามกิ่ง

ส่วนที่สามคือ leaf node หรือใบ ซึ่งคือคำตัดสินสุดท้าย เป็น class label ที่เราต้องการทำนาย พอถึงใบแล้วจบ ไม่ต้องถามอะไรต่อ

โหนดที่อยู่บนสุด เราเรียกพิเศษว่า root node ครับ เป็นคำถามแรกที่ข้อมูลทุกตัวต้องผ่าน และอย่างที่จะได้เห็นต่อไป การเลือก root ให้ถูก คือเรื่องที่ยากที่สุดของทั้งหมด

สไลด์ 4 — โมเดลมี 2 เฟส

เวลาประมาณ 0:50 นาที · สะสม 3:20

โมเดลตัวนี้มีสองเฟสที่ต้องแยกให้ออกนะครับ

เฟสแรกคือ Construction บางตำราเรียก Induction เราป้อนข้อมูลที่รู้คำตอบอยู่แล้ว เรียกว่า training data เข้าไป แล้วให้อัลกอริทึมงอกต้นไม้ขึ้นมาเอง เฟสนี้แหละครับที่ต้องคำนวณหนัก และเป็นเนื้อหาเกือบทั้งหมดของคาบนี้

เฟสที่สองคือ Prediction เอาข้อมูลใหม่ที่ยังไม่รู้คำตอบ มาไหลลงจาก root ตามคำตอบของมันเอง จนตกถึงใบ ค่าที่ใบคือคำทำนาย เฟสนี้ไม่มีการคำนวณเลยครับ เร็วมาก นี่คือเหตุผลที่ decision tree ถูกใช้ในระบบที่ต้องตอบสนองเร็ว

ส่วนสามคำด้านล่างนี้ ให้จำไว้ เพราะมันคือวิธีคิดของอัลกอริทึมทั้งหมด top-down เริ่มจากบนลงล่าง, recursive ทำซ้ำกับทุกกิ่ง, และ divide and conquer แบ่งปัญหาใหญ่เป็นก้อนเล็กแล้วเอาชนะทีละก้อน

สไลด์ 5 — ชุดข้อมูลฝึกสอน

เวลาประมาณ 1:10 นาที · สะสม 4:30

นี่คือข้อมูลทั้งหมดที่เราจะใช้ครับ ทุกตัวเลขในคาบนี้มาจากตารางนี้ตารางเดียว

มี 14 แถว หรือเรียกว่า 14 tuples แต่ละแถวคือลูกค้าหนึ่งคน มี attribute สี่ตัวที่เราใช้ถามได้ คือ age, income, student และ credit_rating

ส่วนคอลัมน์สุดท้าย buys_computer คือ class label หรือคำตอบที่เราอยากทำนาย มีสองค่าคือ yes กับ no เราจึงเรียกปัญหานี้ว่า binary classification

▸ **กดปุ่ม age เพื่อระบายสีตารางตามช่วงอายุ**

ลองระบายสีตาม age ดูนะครับ จะเห็นว่าข้อมูลถูกแบ่งเป็นสามกองตามช่วงอายุ

ทีนี้ดูภาพรวมทางขวา ทั้งหมด 14 แถว เป็น yes 9 แถว no 5 แถว ยังปนกันอยู่มากครับ

ลองคิดตามนะ ถ้าตอนนี้มีลูกค้าใหม่เดินเข้ามา แล้วเราไม่รู้อะไรเลยเกี่ยวกับเขา เราจะเดาว่าอะไร เราก็ต้องเดา yes เพราะเป็นฝั่งที่มากกว่า แล้วเราจะผิดประมาณ 5 ใน 14 ครั้ง หรือราว 36 เปอร์เซ็นต์

เป้าหมายของการสร้างต้นไม้ คือแบ่งข้อมูลไปเรื่อย ๆ จนแต่ละกองเหลือคลาสเดียว เพื่อจะได้ไม่ต้องเดาอีกต่อไป

สไลด์ 6 — ควรถามอะไรก่อน

เวลาประมาณ 0:50 นาที · สะสม 5:20

ทีนี้มาถึงคำถามที่ยากที่สุดครับ

เรามี attribute สี่ตัว ถามได้หมดทุกตัว ไม่มีตัวไหนผิดกฎ แต่ผลลัพธ์ต่างกันลิบ

ถ้าเราเลือกคำถามดี แบ่งแล้วแต่ละกองจะเอนไปทางคลาสเดียวอย่างชัดเจน อย่างแถบด้านซ้ายนี้ กองแรกเขียวล้วน กองที่สองแดงเกือบหมด กองที่สามเขียวเกือบหมด ผลคือต้นไม้เตี้ย ถามน้อยครั้ง และแม่นยำ

แต่ถ้าเลือกคำถามแย่ แบ่งแล้วทุกกองยังคงปนกันครึ่งต่อครึ่งเหมือนเดิม เท่ากับเราเสียคำถามไปหนึ่งข้อโดยไม่ได้ข้อมูลอะไรเพิ่มเลย ต้นไม้ก็จะใหญ่ขึ้นโดยเปล่าประโยชน์

คำถามคือ เราจะรู้ล่วงหน้าได้อย่างไรว่าคำถามไหนดี คำถามไหนแย่ คำตอบคือเราต้องมีตัวเลขที่บอกว่า คำถามหนึ่งลดความมั่วลงได้กี่บิต ตัวเลขนั้นชื่อ Information Gain ครับ

สไลด์ 7 — Info(D) วัดความไม่แน่นอนของข้อมูล

เวลาประมาณ 1:40 นาที · สะสม 7:00

ก่อนจะวัดว่าคำถามลดความมั่วได้เท่าไร เราต้องวัดความมั่วให้เป็นตัวเลขก่อน เครื่องมือนั้นคือ Info of D ครับ

Info of D คือปริมาณความไม่แน่นอนของข้อมูลกอง D มีหน่วยเป็นบิต ถ้าได้ศูนย์ แปลว่ากองนั้นบริสุทธิ์ มีคลาสเดียว ไม่ต้องเดาเลย ถ้าได้หนึ่ง แปลว่าปนกันครึ่งต่อครึ่งพอดี เดาไม่ได้เลย เหมือนโยนเหรียญ

ดูสูตรด้านบนนะครับ Info of D เท่ากับ ลบ ผลรวมของ p ตัวที่ i คูณ log ฐานสองของ p ตัวที่ i โดยที่ p ตัวที่ i คือสัดส่วนของคลาส i ในกองนั้น

ที่ต้องมีเครื่องหมายลบข้างหน้า เพราะ p เป็นค่าระหว่างศูนย์ถึงหนึ่งเสมอ ทำให้ log ของมันติดลบเสมอ ใส่ลบเข้าไปเพื่อให้ผลลัพธ์ออกมาเป็นบวก ไม่มีความหมายลึกซึ้งไปกว่านั้นครับ

ทีนี้มาแทนค่าจริง ข้อมูลเรามี yes 9 no 5 จากทั้งหมด 14 ดังนั้น p ของ yes เท่ากับ 9 ส่วน 14 ได้ 0.6429 และ p ของ no เท่ากับ 5 ส่วน 14 ได้ 0.3571

แทนลงไปในสูตร จะได้ ลบ 9 ส่วน 14 คูณ log สองของ 9 ส่วน 14 ลบอีก 5 ส่วน 14 คูณ log สองของ 5 ส่วน 14 คำนวณออกมาได้ 0.4098 บวก 0.5305 รวมเป็น 0.940 บิต

ดูแถบสเกลทางขวาครับ ค่า 0.940 อยู่เกือบสุดฝั่งสีแดง แปลว่าข้อมูลตั้งต้นของเรา เกือบมั่วที่สุดเท่าที่ข้อมูลสองคลาสจะมั่วได้แล้ว

เลข 0.940 นี้สำคัญมากนะครับ จำไว้ให้ดี เพราะมันคือความไม่รู้เริ่มต้น ที่เราจะพยายามลดลงให้ได้มากที่สุดในทุกขั้นตอนต่อจากนี้

สไลด์ 8 — แบ่งด้วย A แล้วเหลือความไม่แน่นอนเท่าไร

เวลาประมาณ 1:10 นาที · สะสม 8:10

ขั้นต่อไปครับ สมมติเราเลือกถามด้วย attribute ตัวหนึ่ง เรียกมันว่า A

พอถามแล้ว ข้อมูลจะถูกแยกออกเป็นกอง ๆ ตามค่าของ A แต่ละกองมีความมั่วของตัวเองไม่เท่ากัน คำถามคือแล้วเราจะสรุปเป็นตัวเลขเดียวได้อย่างไร คำตอบคือถัวเฉลี่ยแบบถ่วงน้ำหนักครับ

ดูสูตรนะครับ Info A of D เท่ากับผลรวมของ ขนาดกองที่ j หารด้วยขนาดทั้งหมด คูณด้วย Info ของกองที่ j ก้อนแรกคือน้ำหนัก ก้อนที่สองคือความมั่วของกองนั้น

ทำไมต้องถ่วงน้ำหนักครับ ลองนึกว่าเราแบ่งได้สองกอง กองหนึ่งมี 12 แถว อีกกองมี 2 แถว ถ้าเราเอาความมั่วของสองกองมาบวกกันแล้วหารสอง เท่ากับเราให้กองที่มีแค่ 2 แถว มีสิทธิ์ออกเสียงเท่ากับกองที่มี 12 แถว ซึ่งไม่ยุติธรรม เราจึงต้องชั่งด้วยสัดส่วนจำนวนตัวอย่างเสมอ

ใครเคยเรียน information theory มาแล้ว ค่านี้ก็คือ conditional entropy นั่นเองครับ คือความไม่แน่นอนที่ยังเหลืออยู่ หลังจากที่เราได้รู้ค่าของ A แล้ว

สไลด์ 9 — Information Gain

เวลาประมาณ 1:10 นาที · สะสม 9:20

พอเรามีสองตัวเลขนี้แล้ว ที่เหลือก็ง่ายมากครับ Gain of A เท่ากับ Info of D ลบด้วย Info A of D

แปลเป็นภาษาคนคือ เอาความไม่แน่นอนก่อนถาม ลบด้วยความไม่แน่นอนที่ยังเหลือหลังถาม ส่วนต่างที่ได้คือกำไรที่เราได้จากการถามคำถามนั้น มีหน่วยเป็นบิตเหมือนกัน

แล้วกฎของ ID3 กับ C4.5 ก็ตรงไปตรงมาครับ ที่ทุกโหนด ให้คำนวณ Gain ของ attribute ที่เหลือทุกตัว แล้วเลือกตัวที่ได้ Gain สูงที่สุดมาเป็นคำถามของโหนดนั้น เราเรียกวิธีเลือกแบบนี้ว่า greedy คือเลือกสิ่งที่ดีที่สุดเฉพาะหน้าไปทีละก้าว

ขอให้สังเกตสามข้อด้านล่างนะครับ ข้อแรก Gain จะมากกว่าหรือเท่ากับศูนย์เสมอ เพราะการรู้ข้อมูลเพิ่ม ไม่มีทางทำให้เรามั่วขึ้น อย่างมากก็เท่าเดิม

ข้อสอง ถ้า Gain เท่ากับศูนย์พอดี แปลว่าแบ่งแล้วสัดส่วน yes ต่อ no ในทุกกองยังเหมือนเดิม ถามไปก็ไม่ได้อะไร

ข้อสาม ถ้า Gain เท่ากับ Info of D พอดี นั่นคือกรณีสมบูรณ์แบบ แบ่งครั้งเดียวทุกกองบริสุทธิ์หมด ไม่ต้องถามอะไรต่ออีก เดี๋ยวเราจะได้เจอกรณีนี้จริง ๆ ในอีกไม่กี่สไลด์

สไลด์ 10 — ขั้นที่ 1 แบ่งด้วย age

เวลาประมาณ 1:10 นาที · สะสม 10:30

เอาล่ะครับ ลงมือคำนวณจริง เราจะเริ่มจาก attribute ตัวแรกคือ age

พอแบ่ง 14 แถวด้วย age เราจะได้สามกอง

กองแรก อายุน้อยกว่าหรือเท่ากับ 30 มี 5 แถว เป็น yes 2 no 3 คำนวณ I ของ 2 กับ 3 ได้ 0.971 ยังมั่วอยู่มากครับ

กองที่สอง อายุ 31 ถึง 40 มี 4 แถว และเป็น yes ทั้งหมดเลย ไม่มี no สักแถว ค่า I ของ 4 กับ 0 เท่ากับศูนย์พอดี นี่คือกองบริสุทธิ์ครับ

กองที่สาม อายุมากกว่า 40 มี 5 แถว เป็น yes 3 no 2 ได้ 0.971 เท่ากับกองแรกพอดี

ขอให้จับตากองกลางไว้ให้ดีนะครับ การที่ age แบ่งแล้วได้กองบริสุทธิ์มาฟรี ๆ หนึ่งกองทันที คือจุดแข็งที่สุดของมัน และเดี๋ยวเราจะเห็นว่าไม่มี attribute ตัวไหนในชุดข้อมูลนี้ทำแบบนั้นได้เลย

สไลด์ 11 — ขั้นที่ 2 ถึง 3 หา Gain ของ age

เวลาประมาณ 1:10 นาที · สะสม 11:40

ต่อครับ เอาความมั่วของสามกองนั้นมาถัวเฉลี่ยแบบถ่วงน้ำหนัก

กองแรกมี 5 จาก 14 แถว น้ำหนักคือ 5 ส่วน 14 คูณกับ 0.971 กองกลาง 4 ส่วน 14 คูณกับศูนย์ ซึ่งเป็นศูนย์ทั้งก้อน ส่วนกองที่สาม 5 ส่วน 14 คูณกับ 0.971 อีกครั้ง

บวกกันได้ 0.347 บวก ศูนย์ บวก 0.347 เท่ากับ 0.694

เห็นไหมครับว่ากองกลางที่บริสุทธิ์ ช่วยดึงค่าเฉลี่ยลงมาได้เยอะมาก เพราะมันเติมศูนย์เข้ามาหนึ่งก้อนเต็ม ๆ

ทีนี้หา Gain ก็แค่เอา 0.940 ลบ 0.694 ได้ 0.247 บิต แปลว่าการถามว่าอายุเท่าไร ซื้อข้อมูลให้เรามาได้ 0.247 บิต

มีเรื่องหนึ่งต้องเตือนไว้ครับ ในตำราหรือสไลด์บางเล่มจะเขียนว่า 0.246 ไม่ใช่ 0.247 สาเหตุคือเขาเอาค่าที่ปัดเศษแล้วมาลบกัน คือ 0.940 ลบ 0.694 ส่วนของเราคำนวณเต็มความละเอียดแล้วค่อยปัดตอนท้าย ถูกทั้งคู่ครับ และไม่กระทบลำดับการเลือกเลย ซึ่งเป็นสิ่งเดียวที่อัลกอริทึมสนใจ เวลาสอบให้ยึดวิธีที่ผู้สอนกำหนด แต่ต้องแสดงวิธีทำให้ครบเสมอ

สไลด์ 12 — คำนวณครบทั้ง 4 attribute

เวลาประมาณ 1:30 นาที · สะสม 13:10

ทีนี้เราทำแบบเดียวกันกับอีกสามตัวที่เหลือ ผลออกมาเป็นแบบนี้ครับ age ได้ 0.247 สูงสุด รองลงมาคือ student ได้ 0.152 แล้วก็ credit_rating ได้ 0.048 ส่วน income ได้ 0.029 น้อยที่สุด

▸ **กดปุ่ม income**

ลองดู income ก่อนนะครับ ว่าทำไมถึงได้น้อยที่สุด สังเกตว่าทั้งสามกอง high medium low ค่า I ยังสูงใกล้ ๆ ของเดิมหมดเลย อยู่ระหว่าง 0.81 ถึง 1.00 แปลว่าไม่ว่าลูกค้าจะรายได้สูงหรือต่ำ สัดส่วนคนซื้อกับไม่ซื้อก็พอ ๆ กัน income จึงแทบไม่ช่วยแยกอะไรเลย

▸ **กดปุ่ม student**

ส่วน student ได้อันดับสอง เพราะกอง yes เอนไปทางซื้อค่อนข้างชัด คือ 6 ต่อ 1 แต่กอง no ยังมั่วอยู่ที่ 3 ต่อ 4 เลยดึงค่าเฉลี่ยขึ้นไป

▸ **กดปุ่ม credit_rating**

credit_rating น่าสนใจครับ กอง excellent คือ 3 ต่อ 3 พอดี ซึ่ง entropy เท่ากับ 1.0 เต็ม แย่ที่สุดเท่าที่จะเป็นไปได้ เลยทำให้ Gain รวมต่ำ

ดังนั้นผู้ชนะรอบนี้คือ age ครับ ชนะขาด สูงกว่าอันดับสองเกือบเท่าตัว

สไลด์ 13 — age ได้เป็น Root Node

เวลาประมาณ 0:40 นาที · สะสม 13:50

สรุปคือ age ได้เป็น root node ครับ คำถามแรกสุดของต้นไม้คือ อายุอยู่ช่วงไหน

แตกออกมาสามกิ่ง กิ่งกลางคือ 31 ถึง 40 จบเป็นใบได้เลยเพราะบริสุทธิ์แล้ว ส่วนอีกสองกิ่งยังมีค่าความมั่ว 0.971 อยู่ ต้องถามต่อ

ทีนี้ประเด็นสำคัญครับ แต่ละกองที่เหลือ คือปัญหาเดิมในขนาดที่เล็กลง เราจึงทำขั้นตอนเดิมซ้ำกับมันได้เลย นี่แหละครับคือความหมายของคำว่า recursive

มีกติกาข้อเดียวที่ต้องจำ attribute ที่ใช้ไปแล้วบนเส้นทางนั้น จะไม่ถูกนำมาพิจารณาซ้ำ ในที่นี้แต่ละกิ่งจึงเหลือให้เลือกแค่สามตัว คือ income, student และ credit_rating

สไลด์ 14 — กิ่งที่ 1 อายุน้อยกว่าหรือเท่ากับ 30

เวลาประมาณ 0:55 นาที · สะสม 14:45

มาดูกิ่งแรกครับ อายุน้อยกว่าหรือเท่ากับ 30 มี 5 แถว yes 2 no 3 ค่า Info เท่ากับ 0.971

พอคำนวณ Gain ของสามตัวที่เหลือ ได้ผลแบบนี้ student ชนะขาดที่ 0.971 income ได้ 0.571 ส่วน credit_rating ได้แค่ 0.020

ดูตัวเลขของ student ให้ดีนะครับ 0.971 ซึ่งเท่ากับ Info ของโหนดนี้พอดีเป๊ะ จำที่ผมบอกไว้ตอนสไลด์ Gain ได้ไหมครับ ว่าถ้า Gain เท่ากับ Info of D แปลว่ากรณีสมบูรณ์แบบ แบ่งครั้งเดียวแล้วบริสุทธิ์หมด

ดูวิธีคำนวณทางซ้าย student เท่ากับ no มี 3 แถว เป็น no ทั้งหมด I เท่ากับศูนย์ ส่วน student เท่ากับ yes มี 2 แถว เป็น yes ทั้งหมด I เท่ากับศูนย์เช่นกัน ถัวเฉลี่ยอย่างไรก็ได้ศูนย์ แล้ว 0.971 ลบศูนย์ ก็คือ 0.971

ทั้งสองกองจึงกลายเป็นใบทันที ไม่ต้องถามต่อครับ

สไลด์ 15 — กิ่งที่ 2 อายุ 31 ถึง 40

เวลาประมาณ 0:40 นาที · สะสม 15:25

กิ่งที่สองง่ายที่สุดครับ อายุ 31 ถึง 40 มีสี่แถว ดูตารางทางซ้ายได้เลย เป็น yes ทั้งสี่แถว

ข้อสำคัญคือ ก่อนจะคำนวณอะไรทั้งสิ้น ให้ตรวจเงื่อนไขหยุดก่อนเสมอ และโหนดนี้เข้าเงื่อนไขข้อแรกพอดี คือทุก tuple อยู่คลาสเดียวกัน

เมื่อบริสุทธิ์แล้ว Info เท่ากับศูนย์ ไม่มีความไม่แน่นอนเหลือให้ลดอีก ต่อให้เราขยันคำนวณ Gain ของ attribute ทุกตัว ก็จะได้ศูนย์ทั้งหมด เพราะศูนย์ลบศูนย์ก็เท่ากับศูนย์

โหนดนี้จึงจบเป็นใบ ที่ label เท่ากับ Buy ครับ

สไลด์ 16 — กิ่งที่ 3 อายุมากกว่า 40

เวลาประมาณ 0:40 นาที · สะสม 16:05

กิ่งที่สาม อายุมากกว่า 40 มี 5 แถว yes 3 no 2 ค่า Info เท่ากับ 0.971 เท่ากับกิ่งแรกพอดี ต้องหาผู้ชนะกันอีกครั้ง

รอบนี้ credit_rating ชนะครับ ได้ 0.971 เต็มเหมือนเดิม คือแบ่งแล้วบริสุทธิ์ทั้งสองกอง fair เป็น Buy ทั้ง 3 แถว ส่วน excellent เป็น Not-buy ทั้ง 2 แถว

สิ่งที่อยากให้สังเกตคือ credit_rating ตัวนี้ ตอนอยู่ที่โหนด root ได้แค่ 0.048 เป็นอันดับสาม แต่พอมาอยู่โหนดนี้กลับชนะขาด

นี่คือประเด็นสำคัญครับ attribute ที่ดีหรือไม่ดี ไม่ใช่เรื่องตายตัว มันขึ้นอยู่กับว่าเรากำลังยืนอยู่ที่โหนดไหน อัลกอริทึมจึงต้องคำนวณใหม่ทุกครั้งที่โหนด ไม่ใช่จัดอันดับครั้งเดียวแล้วใช้ตลอดทั้งต้น

สไลด์ 17 — ต้นไม้ที่ได้

เวลาประมาณ 0:55 นาที · สะสม 17:00

และนี่คือต้นไม้ที่ได้จากการเรียนรู้ครับ

ตอนนี้ทุกกิ่งกลายเป็นใบหมดแล้ว ไม่มีโหนดไหนที่ยังปนคลาสกันอยู่ recursion จึงยุติลง

ต้นไม้ลึกแค่สองระดับ แปลว่าอย่างมากที่สุด เราถามแค่สองคำถามก็ตัดสินใจได้แล้ว

มีเรื่องหนึ่งที่อยากให้สังเกตเป็นพิเศษครับ ต้นไม้นี้ใช้ attribute แค่ 3 จาก 4 ตัว income ไม่ปรากฏอยู่ในต้นไม้เลยสักโหนดเดียว เพราะมันไม่เคยชนะที่โหนดไหนเลย

นี่คือคุณสมบัติที่เรียกว่า feature selection อัตโนมัติครับ attribute ที่ไม่ให้ข้อมูล จะถูกคัดทิ้งไปเองโดยที่เราไม่ต้องสั่ง ซึ่งเป็นข้อได้เปรียบสำคัญของ decision tree

---

สไลด์ 18 — แปลงต้นไม้เป็นกฎ IF-THEN

เวลาประมาณ 0:40 นาที · สะสม 17:40

ต้นไม้ที่ได้ แปลงเป็นชุดกฎ IF-THEN ได้ตรง ๆ เลยครับ

วิธีนับง่ายมาก หนึ่งเส้นทางจาก root ถึงใบ เท่ากับหนึ่งกฎ ต้นนี้มี 5 ใบ ก็ได้ 5 กฎ

ลองอ่านกฎข้อที่สามดูนะครับ ถ้าอายุ 31 ถึง 40 แล้วซื้อ จบเลย ไม่ต้องดูอย่างอื่น สั้นที่สุด เพราะมันอยู่ลึกแค่ระดับเดียว

ความสามารถตรงนี้คือจุดแข็งที่สุดของ decision tree ครับ เราอธิบายให้ผู้ใช้งานฟังได้เลยว่าโมเดลตัดสินใจแบบนี้เพราะอะไร ต่างจาก neural network ที่บอกได้แค่ผลลัพธ์แต่อธิบายเหตุผลไม่ได้ ในงานที่ต้องรับผิดชอบต่อการตัดสินใจ เช่น การแพทย์หรือการเงิน คุณสมบัตินี้มีค่ามาก

สไลด์ 19 — ทดลองทำนายข้อมูลใหม่

เวลาประมาณ 1:10 นาที · สะสม 18:50

ปิดท้ายด้วยการใช้งานจริงครับ กลับมาที่เฟสที่สอง คือ Prediction

สมมติมีลูกค้าใหม่เดินเข้ามา อายุ 28 ปี รายได้ต่ำ เป็นนักเรียน และเครดิตระดับ excellent เราไม่รู้ว่าเขาจะซื้อหรือไม่ซื้อ วิธีทำนายคือปล่อยให้ข้อมูลไหลลงต้นไม้ครับ

เริ่มที่ root ถามว่า age เท่าไร ได้น้อยกว่าหรือเท่ากับ 30 เพราะเขาอายุ 28 เดินกิ่งซ้าย มาถึงโหนด student ถามว่าเป็นนักเรียนไหม ได้ yes เดินกิ่งขวา ตกถึงใบ ได้คำตอบว่า Buy ครับ

สังเกตนะครับว่าเราใช้คำถามแค่ 2 ครั้ง และไม่ได้ใช้ income กับ credit_rating เลย ทั้งที่เรามีข้อมูลสองตัวนั้นอยู่ในมือ เพราะบนเส้นทางนี้มันไม่จำเป็น

▸ **เปลี่ยนช่อง age เป็น มากกว่า 40**

ลองเปลี่ยนดูนะครับ ถ้าลูกค้าคนเดิมอายุมากกว่า 40 เส้นทางจะเปลี่ยนไปทางขวาทันที และคราวนี้ credit_rating จะกลายเป็นตัวตัดสินแทน โดยที่ student ไม่ถูกใช้เลย

สไลด์ 20 — สรุป และก้าวต่อไป

เวลาประมาณ 1:10 นาที · สะสม 20:00

สรุปทั้งคาบนะครับ เราใช้ของแค่สามอย่าง

หนึ่ง Info of D ใช้วัดความไม่แน่นอนของข้อมูลกองหนึ่ง ยิ่งต่ำยิ่งบริสุทธิ์ สอง Information Gain คือ Info of D ลบ Info A of D ใช้วัดว่าคำถามไหนคุ้มที่สุด แล้วเลือกตัวที่สูงสุด สาม Recursion ทำซ้ำแบบเดิมกับทุกกิ่ง จนกว่าจะเข้าเงื่อนไขหยุด

แต่วิธีนี้มีจุดอ่อนที่ต้องรู้ไว้ครับ Information Gain ลำเอียงเข้าหา attribute ที่มีค่าเป็นไปได้เยอะ ๆ

ลองนึกภาพว่าเราเผลอใส่คอลัมน์ customer_id เข้าไปในข้อมูล ลูกค้า 14 คน ก็มี id 14 ค่าไม่ซ้ำกันเลย พอแบ่งด้วย id จะได้ 14 กอง กองละ 1 แถว บริสุทธิ์หมดทุกกอง Gain สูงปรี๊ดชนะทุกตัว แต่ต้นไม้ที่ได้ทำนายลูกค้าคนใหม่ไม่ได้เลยสักคน เพราะ id ของคนใหม่ไม่เคยอยู่ในข้อมูลฝึก

ทางแก้มีสามทางที่จะได้เรียนกันต่อครับ Gain Ratio ของ C4.5 ที่หารด้วย SplitInfo เพื่อลงโทษ attribute ที่แตกกิ่งเยอะ, Gini Index ของ CART ที่ใช้การวัดความไม่บริสุทธิ์อีกแบบและแตกกิ่งสองทางเสมอ, และ Pruning คือการตัดกิ่งทิ้งเพื่อแก้ปัญหา overfitting

แต่ทั้งสามเรื่องนั้น ตั้งอยู่บนแนวคิดเดียวกับที่เราเพิ่งเรียนจบไปวันนี้ครับ คือเลือกคำถามที่ลดความไม่แน่นอนได้มากที่สุด ขอบคุณครับ